Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ciencias de la Computación

Computación Paralela

Laboratorio PCAM y OpenMP

Simulación gravitacional N-Body en 2D

nina

1 Diseño PCAM y estrategia OpenMP

Se implementaron tres versiones en C++17. La secuencial recorre todos los cuerpos y sus vecinos. La paralela directa reparte cuerpos entre hilos. La optimizada calcula cada pareja una sola vez y suma fuerzas iguales con sentidos opuestos.

Se utilizó la semilla 42, posiciones entre -1 y 1, masas entre 1 y 2 y velocidades iniciales cero. La constante gravitacional fue 0.001, el paso temporal 0.001 y el suavizado cuadrático 0.0001. Se actualiza primero la velocidad y luego la posición. El suavizado evita una división entre cero cuando dos cuerpos coinciden.

1.1 Partición

La unidad mínima es una interacción entre dos cuerpos. Crear una tarea por interacción tendría un costo alto frente al poco cálculo que contiene. En la directa se agrupan todas las interacciones de un cuerpo. En la optimizada se reparte una fila de parejas con índice mayor. Estas filas tienen tamaños distintos. Un paso necesita las posiciones completas del paso anterior, por lo que los pasos no se pueden ejecutar al mismo tiempo.

1.2 Comunicación y carreras

Las posiciones y masas son compartidas y se leen durante el cálculo de fuerzas. En la directa cada cuerpo tiene un solo escritor para su fuerza. En la optimizada dos parejas pueden sumar sobre el mismo cuerpo y producir una carrera. Se resolvió con un vector de fuerzas privado por hilo. Al terminar las parejas se combinan esos vectores por cuerpo y en orden de hilo. No se usan atomic ni critical y la memoria adicional crece con el número de cuerpos y de hilos.

1.3 Aglomeración y mapeo

Se compararon chunks de 8, 32 y 64 cuerpos. Los grupos pequeños dan más oportunidades para repartir trabajo, pero aumentan las asignaciones. Los grandes reducen ese costo y pueden dejar carga desigual. Static asigna bloques de antemano, dynamic entrega nuevos bloques cuando un hilo queda libre y guided empieza con bloques grandes que disminuyen. La optimizada necesita más cuidado porque las primeras filas contienen más parejas.

1.4 Sincronización planeada

Todos los ciclos OpenMP usan nowait. Se escribieron barreras explícitas después del cálculo, después de combinar acumuladores y después de actualizar posiciones. En la optimizada también se separa la limpieza de acumuladores del cálculo. Se mantiene una región paralela durante todos los pasos. El cierre de esa región tiene una sincronización obligatoria de OpenMP que no se puede eliminar; las dependencias ya quedan resueltas por las barreras explícitas.

2 Mediciones y resultados

Se usaron 5000 cuerpos, 10 pasos y tres ejecuciones por configuración. Se guardaron 63 mediciones en results.csv y los promedios en summary.csv. Se compiló con GCC 10.3.0, optimización O2 y OpenMP en Windows 11. OpenMP informó 8 procesadores disponibles, un máximo de 8 hilos y equipos reales de 2, 4 y 8 hilos según cada prueba. Ese dato no distingue núcleos físicos de procesadores lógicos.

El tiempo se midió con omp_get_wtime. Incluye reservar fuerzas y realizar la simulación. La entrada, la impresión y la referencia de corrección quedan fuera de ese intervalo. El promedio secuencial fue 0.6317 segundos. Se calculó $S = T_1/T_p$ y $E = S/p$. Default indica el chunk predeterminado del runtime. La eficiencia se muestra como porcentaje.

2.1 Versión paralela directa

Hilos	Reparto	Chunk	Tiempo s	Speedup	Eficiencia
2	static	default	0.4497	1.405	70.2%
4	static	default	0.2430	2.599	65.0%
8	static	default	0.1997	3.164	39.5%
8	static	8	0.2433	2.596	32.4%
8	static	32	0.2137	2.956	37.0%
8	static	64	0.2100	3.008	37.6%
8	dynamic	8	0.1940	3.256	40.7%
8	dynamic	32	0.1977	3.196	39.9%
8	dynamic	64	0.1887	3.348	41.9%
8	guided	default	0.1947	3.245	40.6%

2.2 Versión optimizada por parejas

Hilos	Reparto	Chunk	Tiempo s	Speedup	Eficiencia
2	static	default	0.3660	1.726	86.3%
4	static	default	0.2600	2.429	60.7%
8	static	default	0.2227	2.837	35.5%
8	static	8	0.1380	4.577	57.2%
8	static	32	0.1337	4.726	59.1%
8	static	64	0.1240	5.094	63.7%
8	dynamic	8	0.1287	4.909	61.4%
8	dynamic	32	0.1243	5.080	63.5%
8	dynamic	64	0.1223	5.163	64.5%
8	guided	default	0.1703	3.708	46.4%

3 Análisis y conclusiones

Dynamic con chunk 64 obtuvo el menor promedio en ambas versiones: 0.1887 segundos en la directa y 0.1223 segundos en la optimizada. Static con chunk 64 quedó muy cerca en la optimizada, con 0.1240 segundos. Las diferencias pequeñas no bastan para afirmar que una política siempre será mejor. La optimizada con dynamic 64 tardó cerca de 35.2 por ciento menos que la directa con la misma configuración.

En la optimizada, static predeterminado tardó 0.2227 segundos con 8 hilos. Repartir bloques contiguos del triángulo deja más trabajo en los primeros hilos. Los chunks 8, 32 y 64 redujeron ese promedio a 0.1380, 0.1337 y 0.1240 segundos. En esta prueba, agrupar 64 filas mantuvo un buen reparto y redujo el costo de asignación. Guided predeterminado llegó a 0.1703 segundos y no igualó a los mejores grupos explícitos.

Tener menos operaciones no garantizó menor tiempo. Con static predeterminado y 4 hilos, la directa tardó 0.2430 segundos y la optimizada 0.2600. Con 8 hilos ocurrió lo mismo: 0.1997 frente a 0.2227. La reducción de parejas se acompaña de limpieza de vectores, combinación y barreras adicionales. El desbalance también puede superar el ahorro de cálculo.

Con static predeterminado, la directa obtuvo speedups de 1.405, 2.599 y 3.164 para 2, 4 y 8 hilos. Sus eficiencias fueron 70.2, 65.0 y 39.5 por ciento. La optimizada obtuvo 1.726, 2.429 y 2.837, con eficiencias de 86.3, 60.7 y 35.5 por ciento. La escala ya se aleja de lo lineal con 2 hilos y la pérdida es más clara entre 4 y 8. La mejor configuración optimizada alcanzó speedup 5.163 y eficiencia 64.5 por ciento con 8 hilos.

No se espera obtener ocho veces el rendimiento al pasar de 8 a 64 núcleos. Las mediciones ya muestran costos de sincronización y reparto. Además, combinar acumuladores requiere más memoria y más lecturas al aumentar los hilos. No se midió un equipo con 64 núcleos, por lo que no se puede dar una mejora numérica confiable.

3.1 Corrección y límites de la medición

Cada ejecución comparó posiciones y velocidades contra la referencia secuencial. El checksum fue aproximadamente -581400.034189. La directa tuvo diferencia máxima cero y la optimizada llegó a 6.08×10^{-16}. El cambio en el orden de las sumas explica esa diferencia. Se verificaron 144 configuraciones pequeñas y siete entradas inválidas. No se observaron diferencias mayores que la tolerancia de 10^{-9}.

Los tiempos variaron entre repeticiones. Por ejemplo, la directa con 2 hilos estuvo entre 0.391 y 0.542 segundos. Solo se hicieron tres repeticiones y no se fijó afinidad ni se aisló la actividad del sistema. Los resultados permiten comparar esta ejecución, pero no garantizan los mismos valores en otro equipo.

Se logró reducir trabajo sin carreras mediante fuerzas privadas y barreras explícitas. El reparto de parejas tuvo tanto peso como el número de operaciones. Los registros de ejecución se incluyen como imágenes en los anexos y sus archivos de texto conservan los comandos y resultados originales. Estas imágenes representan registros reales, no capturas del escritorio.

Repositorio del laboratorio.

A Evidencia de compilación y ejecución

```
Registro de salida real de terminal

$ g++ -O2 -std=c++17 -fopenmp -Wall -Wextra -pedantic src/nbody_seq.cpp -o build/nbody_seq.exe

Exit code: 0
$ g++ -O2 -std=c++17 -fopenmp -Wall -Wextra -pedantic src/nbody_parallel.cpp -o build/nbody_parallel.exe

Exit code: 0
$ g++ -O2 -std=c++17 -fopenmp -Wall -Wextra -pedantic src/nbody_optimized.cpp -o build/nbody_optimized.exe

Exit code: 0
```

Figura 1: Comandos de compilación y salida sin errores.

```
Registro de salida real de terminal

$ ./build/nbody_seq.exe --n 5000 --steps 10 --threads 1 --schedule static --chunk 0
Student: Nina Najera Marakovits
Executable: ./build/nbody_seq.exe
Mode: sequential
N: 5000 | Steps: 10 | Seed: 42
Threads requested: 1 | Threads used: 1
Schedule: sequential | Chunk: 0
Elapsed: 0.637999773026 s
Checksum: -581400.034189
Max difference: 0
Processors reported by OpenMP: 8
Max threads reported by OpenMP: 8
Exit code: 0
```

Figura 2: Primera ejecución secuencial registrada.

B Evidencia paralela y hardware

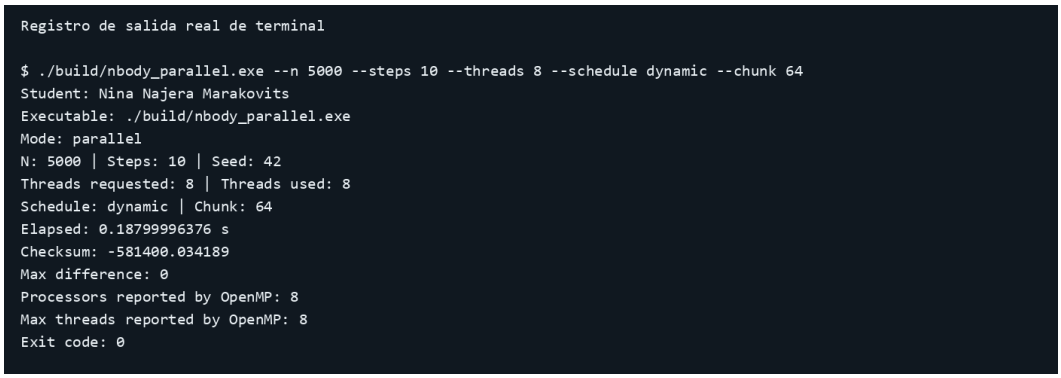


Figura 3: Ejecución directa con 8 hilos y dynamic 64.

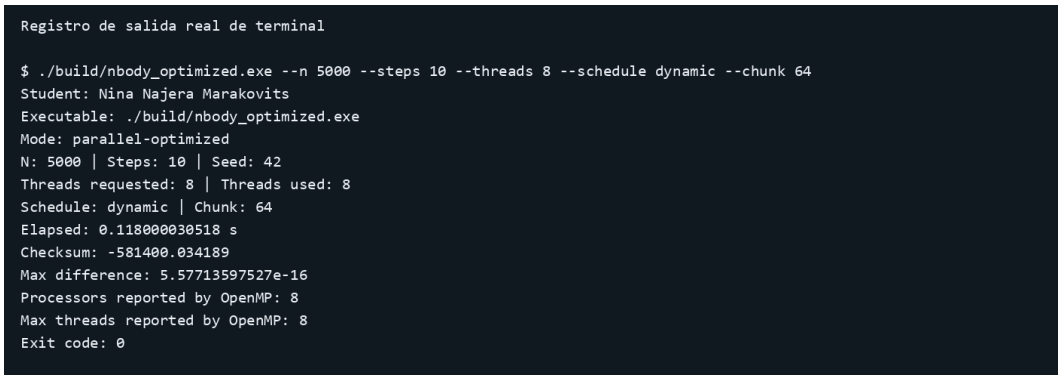


Figura 4: Ejecución por parejas y procesadores disponibles.

C Resumen de hardware

```
Registro de salida real de terminal

$ ./build/nbody_optimized.exe --n 5000 --steps 10 --threads 8 --schedule dynamic --chunk 64
Student: Nina Najera Marakovits
Executable: ./build/nbody_optimized.exe
Mode: parallel-optimized
N: 5000 | Steps: 10 | Seed: 42
Threads requested: 8 | Threads used: 8
Schedule: dynamic | Chunk: 64
Elapsed: 0.118000030518 s
Checksum: -581400.034189
Max difference: 5.57713597527e-16
Processors reported by OpenMP: 8
Max threads reported by OpenMP: 8
Exit code: 0
```

Figura 5: Salida con omp_get_num_procs y omp_get_max_threads.